

PART XII

HEAPS

Min-heaps, max-heaps, heapify, heap sort, and priority_queue in competitive C++

A heap is the data structure behind “give me the smallest/largest thing, repeatedly, as things also get added.” Every time a problem needs a running minimum or maximum, a top-K answer, or an efficient greedy that always looks at “the best option so far,” a heap is one of the first tools to reach for. This Part builds the structure from its array representation up through Dijkstra-style and two-heap patterns you will reuse for the rest of this book.

Contents of Part XII

- 1. What Is a Heap?
- 2. The Array Representation
- 3. Min-Heap vs. Max-Heap
- 4. Insertion (Sift-Up)
- 5. Extracting the Top (Sift-Down)
- 6. Building a Heap From an Array — $O(n)$ Heapify
- 7. Heap Sort
- 8. C++ STL `priority_queue`
- 9. Heap Applications in Competitive Programming
- Part XII Revision, Cheat Sheet, and Interview Questions

Chapter 1 — What Is a Heap?

1.1 The one-sentence definition

In one sentence: a heap is a complete binary tree where every parent is $\leq$ both its children (min-heap) or $\geq$ both its children (max-heap).

That single ordering rule — called the heap property — is the entire idea. It says nothing about how left and right children compare to each other, and nothing about the order of elements at the same level. It only constrains parent vs. child. That weaker-than-sorted guarantee is exactly what makes a heap cheap to maintain: you never need to keep an entire array sorted, you only ever need to repair one root-to-leaf path.

1.2 Why not just keep the array sorted?

You could maintain a fully sorted array and always read the minimum from the front. The problem is insertion and deletion: keeping a sorted array sorted after every insert costs $O(n)$ in the worst case (shifting elements). A heap gives up full sorting and, in exchange, supports both “insert a new element” and “remove the current best element” in $O(\log n)$ — and reading the current best (without removing it) in $O(1)$. That trade — partial order for speed — is why heaps exist.

1.3 “Complete” binary tree, precisely

A binary tree is complete if every level is fully filled except possibly the last, and the last level is filled strictly left to right with no gaps. This is a stronger condition than “balanced,” and it is the property that lets a heap be stored in a plain array with no pointers at all — covered next.

Why this matters

A general binary search tree needs actual node pointers because it can be shaped arbitrarily. A heap's completeness guarantee is what lets it skip pointers entirely and live in a flat array, which is both faster (cache-friendly, no allocation) and simpler to code under contest time pressure.

Chapter 2 — The Array Representation

Because a heap is always a complete binary tree, its shape is fully determined by how many elements it has — there is only one possible shape for n nodes. That means you never need to store the shape at all; you only store the values, in level order, in a plain array.

```
// 0-indexed array representation
// for a node at index i:
int parent(int i) { return (i - 1) / 2; }
int leftChild(int i) { return 2 * i + 1; }
int rightChild(int i) { return 2 * i + 2; }
```

Given index i , its children live at fixed, computable positions — no pointer ever needs to be followed or stored. Walk through a concrete example: the array $[1, 3, 2, 7, 4, 5]$ as a min-heap.

Index	Value	Parent idx	Left child idx	Right child idx
0	1	— (root)	1	2
1	3	0	3	4
2	2	0	5	6 (absent)
3	7	1	7 (absent)	8 (absent)
4	4	1	—	—
5	5	2	—	—

Check the heap property by eye: node 0 (value 1) $\leq$ nodes 1 and 2 (3 and 2). Node 1 (value 3) $\leq$ nodes 3 and 4 (7 and 4). Node 2 (value 2) $\leq$ node 5 (5). Every parent-child pair holds — this is a valid min-heap, even though the array is clearly not fully sorted (7 appears before 4 and 5).

⚠ Common mistake

Confusing a heap with a sorted array is one of the most frequent beginner errors. A heap only guarantees the root is the minimum/maximum — nothing about the relative order of any other pair of elements. Never assume $\text{in}[i]$ and $\text{in}[i+1]$ are ordered.

Chapter 3 — Min-Heap vs. Max-Heap

The only difference between a min-heap and a max-heap is the direction of the comparison used everywhere in the implementation — the algorithms are otherwise identical.

Property	Min-heap	Max-heap
Parent vs. children	parent $\leq$ children	parent $\geq$ children
Root holds	the smallest element	the largest element
Typical use	Dijkstra, k smallest, merge sorted lists	k largest, scheduling by max priority
C++ STL default	priority_queue<int, vector<int>, greater<int>>	priority_queue<int> (default)

A useful trick for a max-heap when you only have min-heap code (or vice-versa): negate every value going in and out. This is common enough in contest code that it's worth remembering rather than rewriting the comparison logic.

```
// simulate a max-heap using a min-heap of negated values
priority_queue<int, vector<int>, greater<int>> minHeap;
minHeap.push(-x); // insert x as if it were a max-heap
int largest = -minHeap.top(); // read the current maximum
```

Chapter 4 — Insertion (Sift-Up)

In one sentence: to insert, place the new value at the very next free array slot (keeping the tree complete), then repeatedly swap it with its parent while it violates the heap property.

This upward-repair walk is called sift-up (or bubble-up / percolate-up). It only ever needs to fix a single root-to-leaf path, which is why it costs $O(\log n)$ — the height of a complete binary tree with n nodes is $\lceil \log_2(n+1) \rceil$.

```
// Min-heap insertion on a 0-indexed vector<int> heap
void siftUp(vector<int> &heap, int i) {
    while (i > 0) {
        int par = (i - 1) / 2;
        if (heap[par] <= heap[i]) break; // property holds, stop
        swap(heap[par], heap[i]);
        i = par;
    }
}

void insert(vector<int> &heap, int value) {
    heap.push_back(value); // step 1: place at the next free slot
    siftUp(heap, heap.size() - 1); // step 2: bubble it up into
    position
}
```

Dry run: insert 0 into the min-heap [1, 3, 2, 7, 4, 5]

Step	Array state	Action
Start	[1, 3, 2, 7, 4, 5]	push_back(0) → [1, 3, 2, 7, 4, 5, 0]
1	i = 6, parent = 2, heap[2]=2 > heap[6]=0	swap → [1, 3, 0, 7, 4, 5, 2]
2	i = 2, parent = 0, heap[0]=1 > heap[2]=0	swap → [0, 3, 1, 7, 4, 5, 2]
3	i = 0, no parent	stop — 0 is now the root

Complexity: $O(\log n)$ time (at most the tree's height in swaps), $O(1)$ extra space.

Chapter 5 — Extracting the Top (Sift-Down)

In one sentence: to remove the root (the current min or max), move the very last element into the root's slot, shrink the array by one, then repeatedly swap it downward with its smaller (min-heap) or larger (max-heap) child while it violates the heap property.

This downward-repair walk is sift-down (bubble-down / percolate-down / heapify-down). Why move the last element to the root instead of, say, promoting a child directly? Because the last element is guaranteed to keep the tree complete — removing any other node would leave a hole in the middle of the array that breaks the shape guarantee the whole structure depends on.

```
// Min-heap extraction on a 0-indexed vector<int> heap
void siftDown(vector<int> &heap, int i) {
    int n = heap.size();
    while (true) {
        int left = 2 * i + 1, right = 2 * i + 2, smallest = i;
        if (left < n && heap[left] < heap[smallest]) smallest = left;
        if (right < n && heap[right] < heap[smallest]) smallest = right;
        if (smallest == i) break; // property holds, stop
        swap(heap[i], heap[smallest]);
        i = smallest;
    }
}

int extractMin(vector<int> &heap) {
    int top = heap[0];
    heap[0] = heap.back(); // step 1: move last element to the root
    heap.pop_back();
    if (!heap.empty()) siftDown(heap, 0); // step 2: bubble it down
    return top;
}
```

Dry run: extractMin on [0, 3, 1, 7, 4, 5, 2]

Step	Array state	Action
Start	[0, 3, 1, 7, 4, 5, 2]	top = 0 (saved). Move last (2) to root, pop_back → [2, 3, 1, 7, 4, 5]
1	i=0: left=3(idx1), right=1(idx2)	smallest child is 1 at idx 2 → swap → [1, 3, 2, 7, 4, 5]
2	i=2: left=5(idx5), right absent	heap[5]=5 > heap[2]=2, property holds → stop
Result	[1, 3, 2, 7, 4, 5]	returned value was 0

Complexity: $O(\log n)$ time, $O(1)$ extra space. Reading the top without removing it (`heap[0]`, or `.top()` in STL) is $O(1)$.

⚠ **Common mistake**

Forgetting to check n on left/right child indices before comparing ($\text{left} < n$ and $\text{right} < n$) causes an out-of-bounds read whenever a node has fewer than two children — which is most nodes in the last level.

Chapter 6 — Building a Heap From an Array — $O(n)$ Heapify

Given an arbitrary array of n values, you could build a heap by inserting them one at a time (n insertions $\times O(\log n)$ each = $O(n \log n)$). There's a faster way.

In one sentence: call sift-down on every non-leaf node, starting from the last non-leaf node and working backward to the root — this builds a valid heap in $O(n)$ total time, not $O(n \log n)$.

```
void buildHeap(vector<int> &heap) {  
    int n = heap.size();  
    for (int i = n / 2 - 1; i >= 0; i--) { // last non-leaf down to  
        the root  
        siftDown(heap, i);  
    }  
}
```

6.1 Why start at $n/2 - 1$?

Every index from $n/2$ to $n-1$ is a leaf (it has no children to violate the property with), so sift-down on a leaf does nothing. $n/2 - 1$ is the last index that actually has at least one child, so it's the first useful place to start.

6.2 Why is this $O(n)$, not $O(n \log n)$?

This is a genuinely subtle and commonly-asked result, worth understanding rather than memorizing. A sift-down's cost is

proportional to the height of the node it starts from, not the height of the whole tree. Most nodes in a heap are near the bottom, where the remaining height — and therefore the sift-down cost — is small.

Level (from bottom, $h=0$)	Approx. node count	Max sift-down cost per node
$h = 0$ (leaves)	$n/2$	0 — skipped entirely
$h = 1$	$n/4$	1 swap
$h = 2$	$n/8$	2 swaps
$h = k$	$n / 2^{(k+1)}$	k swaps

Summing (node count at height h) $\times h$ across all levels gives a series that converges to $O(n)$, not $O(n \log n)$ — because the levels with the most nodes (near the bottom) cost almost nothing, and only the few nodes near the root cost close to $\log n$. This is why `buildHeap` in C++'s STL (`make_heap`) and this book always prefer bottom-up construction over n individual insertions.

♦ Interview favorite

“Why is building a heap $O(n)$ and not $O(n \log n)$?” is one of the most commonly asked heap questions. The answer above — sift-down cost depends on height, and most nodes have small height — is the complete answer; be ready to state it, not just the $O(n)$ result.

Chapter 7 — Heap Sort

Heap sort combines Chapters 5 and 6: build a max-heap in $O(n)$, then repeatedly extract the maximum and place it at the end of the array, shrinking the heap by one each time.

```

void siftDownMax(vector<int> &a, int i, int n) { // max-heap
sift-down, bounded by n
    while (true) {
        int left = 2 * i + 1, right = 2 * i + 2, largest = i;
        if (left < n && a[left] > a[largest]) largest = left;
        if (right < n && a[right] > a[largest]) largest = right;
        if (largest == i) break;
        swap(a[i], a[largest]);
        i = largest;
    }
}

void heapSort(vector<int> &a) {
    int n = a.size();
    for (int i = n / 2 - 1; i >= 0; i--) siftDownMax(a, i, n); //
build max-heap
    for (int end = n - 1; end > 0; end--) {
        swap(a[0], a[end]); // move current max to its final sorted
position
        siftDownMax(a, 0, end); // repair the heap over the shrunk range
[0, end)
    }
}

```

7.1 Why in-place, and why not stable

Heap sort needs no extra array — the heap lives in the same array being sorted, and the sorted suffix grows in the space vacated at the end. That makes it in-place: $O(1)$ auxiliary space. It is not stable: the swaps performed during sift-down can reorder two equal elements relative to each other, and there is no way to fix this without extra bookkeeping.

Property	Heap sort
Time complexity (all cases)	$O(n \log n)$
Space complexity	$O(1)$ — in-place
Stable?	No
When it's useful	Guaranteed $O(n \log n)$ with $O(1)$ space, no worst-case blowup like quicksort

Property	Heap sort
When it's not the first choice	std::sort is a highly-tuned introsort and is almost always faster in practice; use library sort unless the problem specifically needs heap-sort's guarantees
♦ Practical rule In a contest, you will almost never hand-write heap sort to sort an array — sort() is faster and shorter. Heap sort matters here because it cements how sift-down and buildHeap work; the priority_queue in the next chapter is what you'll actually reach for.	

Chapter 8 — C++ STL priority_queue

priority_queue is the STL's heap: it hides the array, the indices, and the sift-up/sift-down calls behind push, pop, and top.

```
priority_queue<int> maxHeap; // max-heap by default
maxHeap.push(5); maxHeap.push(1); maxHeap.push(9);
cout << maxHeap.top(); // 9 – O(1) peek
maxHeap.pop(); // removes 9 – O(log n)

priority_queue<int, vector<int>, greater<int>> minHeap; //
min-heap
minHeap.push(5); minHeap.push(1); minHeap.push(9);
cout << minHeap.top(); // 1
```

8.1 Reading the min-heap declaration

priority_queue takes three template arguments: the element type, the underlying container (vector by default), and the comparator. Passing greater<int> flips the comparison so the smallest element sorts to the top — the same trick you'll see in sort() for descending order.

Operation	Meaning	Complexity
.push(x)	insert x	O(log n)
.pop()	remove the top element	O(log n)

Operation	Meaning	Complexity
.top()	read the top element without removing it	O(1)
.empty()	check if the heap has no elements	O(1)
.size()	number of elements	O(1)

8.2 priority_queue of pairs and custom comparators

In competitive programming, `priority_queue` almost always holds pair or a small struct — typically (priority, identifier) — so the queue orders by priority but you can still recover which item it belongs to.

```
// max-heap of (distance, node) pairs – pair compares first
// element first,
// so this orders by distance automatically
priority_queue<pair<int,int>> pq;
pq.push({10, 2}); // distance 10, node 2
pq.push({3, 5}); // distance 3, node 5
auto [dist, node] = pq.top(); // {10, 2} – largest distance first

// custom comparator: min-heap of pairs ordered by first element
// ascending
priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> minpq;

// lambda-based custom comparator (needs decltype since the
// lambda has no name)
auto cmp = [](pair<int,int> a, pair<int,int> b) {
    return a.second > b.second; // order by SECOND element ascending
};
priority_queue<pair<int,int>, vector<pair<int,int>>,
decltype(cmp)> pq2(cmp);
```

⚠ **Common mistake**

A lambda comparator for `priority_queue` must be passed both as a template argument (via `decltype`) and as a constructor argument — forgetting the constructor argument is a frequent compile error. Also remember: the comparator convention is reversed from what feels natural — returning true for “a should come before b” in `sort()` means “a has lower priority than b” in `priority_queue`, since the queue is a max-heap by the same ordering.

Chapter 9 — Heap Applications in Competitive Programming

9.1 K largest / K smallest elements

To find the K largest elements of a stream or array without sorting everything: maintain a min-heap of size K. Push each new element; if the heap exceeds size K, pop the minimum. At the end, the heap holds exactly the K largest elements, and its top is the smallest of them (the “Kth largest”).

```
int kthLargest(vector<int> &nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap; //
    min-heap, size <= k
    for (int x : nums) {
        minHeap.push(x);
        if ((int)minHeap.size() > k) minHeap.pop(); // evict the smallest
    }
    return minHeap.top(); // the Kth largest
}
```

Complexity: $O(n \log k)$ — far better than sorting the whole array ($O(n \log n)$) when k is small relative to n.

9.2 Merging K sorted lists

Put the first element of each of the K lists into a min-heap, tagged with which list and index it came from. Repeatedly pop the minimum, output it, and push the next element from the same list (if any).

```
// merge k sorted vectors into one sorted vector
vector<int> mergeKSorted(vector<vector<int>> &lists) {
    // {value, listIndex, elementIndex}
    priority_queue<array<int,3>, vector<array<int,3>>,
greater<array<int,3>>> pq;
    for (int i = 0; i < (int)lists.size(); i++)
        if (!lists[i].empty()) pq.push({lists[i][0], i, 0});

    vector<int> result;
    while (!pq.empty()) {
        auto [val, li, ei] = pq.top(); pq.pop();
        result.push_back(val);
        if (ei + 1 < (int)lists[li].size())
            pq.push({lists[li][ei + 1], li, ei + 1});
    }
    return result;
}
```

Complexity: $O(N \log K)$, where N is the total number of elements across all lists and K is the number of lists — each of the N elements is pushed and popped once, and the heap never holds more than K elements.

9.3 Running median with two heaps

In one sentence: keep a max-heap of the smaller half of the numbers seen so far and a min-heap of the larger half, kept balanced in size — the median is then always at one or both tops.

```
priority_queue<int> lowerHalf; // max-heap
priority_queue<int, vector<int>, greater<int>> upperHalf; //
min-heap

void addNumber(int x) {
    if (lowerHalf.empty() || x <= lowerHalf.top()) lowerHalf.push(x);
    else upperHalf.push(x);
    // rebalance so sizes never differ by more than 1
    if (lowerHalf.size() > upperHalf.size() + 1) {
        upperHalf.push(lowerHalf.top()); lowerHalf.pop();
    } else if (upperHalf.size() > lowerHalf.size() + 1) {
        lowerHalf.push(upperHalf.top()); upperHalf.pop();
    }
}

double getMedian() {
    if (lowerHalf.size() == upperHalf.size())
        return (lowerHalf.top() + upperHalf.top()) / 2.0;
    return lowerHalf.size() > upperHalf.size() ? lowerHalf.top() :
upperHalf.top();
}
```

Every `addNumber` call is $O(\log n)$; `getMedian` is $O(1)$. This exact two-heap pattern recurs anywhere you need a running “split point” statistic, not just the median.

9.4 A preview: Dijkstra's shortest path

Dijkstra's algorithm (covered in full in the Graphs Part) is, at its core, repeated “pop the closest unvisited node, relax its neighbors, push any improved distances” — exactly the min-heap pop/push loop from this chapter. If the two-heap median pattern and the K-largest pattern feel natural after this chapter, Dijkstra's use of a heap will feel like the same idea applied to graphs, not a new structure to learn.

♦ Pattern to remember

Any time a problem says “always process the current best/smallest/closest item next, and new candidates keep appearing,” a heap is very likely the intended data structure — whether that's Dijkstra, task scheduling by priority, or a running top-K.

9.5 Common mistakes with heaps in contests

Mistake	Why it happens / fix
Using a max-heap where a min-heap (or vice versa) was needed	Read the problem again — “Kth largest” often wants a min-heap of size K, which surprises beginners expecting a max-heap
Forgetting <code>priority_queue</code> has no way to erase an arbitrary element	Use “lazy deletion”: leave stale entries in place and skip them when popped if a validity check fails
Assuming heap order lets you iterate elements in sorted order directly	You can only safely read the top; iterating the underlying container is not sorted order
Comparator logic reversed for a custom struct	Remember <code>priority_queue</code> 's comparator answers “should a have lower priority than b”, which is the opposite feel from <code>sort()</code> 's “should a come before b” for a min-heap-like ordering
Copying a large struct into the heap on every push	Store only the fields you need to compare and look up (e.g. an index into another array) instead of a heavy object

Practice — Chapter 9

Problem 12.1 — Kth Largest Element [Level 2]

Hint 1: You don't need to sort the whole array.

Key observation: A min-heap of size k always holds the k largest elements seen so far; its top is the k th largest.

Approach: Push each element; if size exceeds k , pop the minimum. Return the top at the end.

Complexity: $O(n \log k)$ time, $O(k)$ space.

Problem 12.2 — Merge K Sorted Arrays [Level 3]

Hint 1: Put one candidate element from each array into a heap.

Key observation: Whichever element is currently smallest across all fronts must be next in the merged output.

Approach: Min-heap of (value, arrayIndex, elementIndex); pop the min, push the next element from the same array.

Complexity: $O(N \log K)$ time, where N is total elements and K is the number of arrays.

Problem 12.3 — Running Median [Level 4]

Hint 1: Think about splitting the numbers seen so far into a “lower half” and “upper half.”

Key observation: If both halves are kept the same size (or off by one), the median is always at a top.

Approach: Max-heap for the lower half, min-heap for the upper half, rebalance after every insertion.

Complexity: $O(\log n)$ per insertion, $O(1)$ per median query.

Problem 12.4 — Last Stone Weight (LeetCode-style) [Level 1]

Hint 1: Repeatedly smash the two heaviest stones together.

Approach: Max-heap of stone weights; pop the two largest, push back their difference if nonzero, repeat until at most one stone remains.

Complexity: $O(n \log n)$ overall.

Problem 12.5 — Connect Ropes with Minimum Cost [Level 2]

Hint 1: Connecting two ropes costs the sum of their lengths; you want the total cost minimized.

Key observation: Always connecting the two currently-shortest ropes first is optimal — a greedy argument similar to Huffman coding.

Approach: Min-heap of rope lengths; repeatedly pop two, push their sum back, add the sum to a running total cost.

Complexity: $O(n \log n)$.

Part XII — Master Revision

Must Remember

- A heap is a complete binary tree with the heap property (parent $\leq$ children for min-heap, $\geq$ for max-heap) — it is NOT a sorted array.
- Array representation: $\text{parent}(i) = (i-1)/2$, $\text{left}(i) = 2i+1$, $\text{right}(i) = 2i+2$. No pointers needed.
- Insertion is sift-up (bubble toward the root); extraction is sift-down (bubble toward the leaves). Both are $O(\log n)$.
- Building a heap from n arbitrary elements is $O(n)$ via bottom-up sift-down, not $O(n \log n)$ via repeated insertion.
- Heap sort is $O(n \log n)$ time, $O(1)$ space, in-place, and NOT stable.
- `priority_queue<int>` is a max-heap by default; `add`, `vector<int>`, `greater<int>>` for a min-heap.
- The “k-th largest” pattern uses a min-heap of size k , which surprises people expecting a max-heap.
- The running-median pattern (two balanced heaps) generalizes to any “running split-point statistic” problem.

Common Mistakes Recap

Mistake	Fix
Forgetting to bound-check left/right child indices against n	Always check $\text{left} < n$ and $\text{right} < n$ before comparing/using
Assuming the heap array is sorted	Only the root is guaranteed to be the min/max; nothing else is ordered
Using a max-heap for a “k smallest” style problem	Re-derive which extreme you need to evict, not which extreme you want
Reversed comparator logic for a custom <code>priority_queue</code> struct	Comparator returns true when a has LOWER priority than b
Trying to erase an arbitrary element from <code>priority_queue</code>	Not supported directly — use lazy deletion or switch to an ordered set/multiset

Mistake	Fix
Rebuilding a heap with n insertions instead of <code>buildHeap</code>	Use the $O(n)$ bottom-up sift-down construction when the full array is known upfront

Complexity Table — Heaps

Operation	Time	Space
Peek top (min/max)	$O(1)$	$O(1)$
Insert	$O(\log n)$	$O(1)$ amortized
Extract top	$O(\log n)$	$O(1)$
Build heap from n elements	$O(n)$	$O(1)$ extra (in-place)
Heap sort	$O(n \log n)$	$O(1)$ extra (in-place)
K largest / K smallest of n elements	$O(n \log k)$	$O(k)$
Merge k sorted lists, N total elements	$O(N \log k)$	$O(k)$

Interview-Style Questions

- 1 Why can a heap be stored in a plain array with no pointers, while a general binary tree usually cannot?
- 2 Why is building a heap from n elements $O(n)$ rather than $O(n \log n)$?
- 3 Why is heap sort not a stable sorting algorithm?
- 4 Why does finding the k largest elements with a min-heap of size k beat sorting the whole array when k is small?
- 5 Why can't you efficiently erase an arbitrary element from a `priority_queue`, and what's a common workaround?
- 6 Why does a min-heap, not a max-heap, best support the “ k -th largest element” pattern?

Cheat Sheet — Part XII

```
// Manual heap essentials (0-indexed array)
parent(i) = (i-1)/2 left(i) = 2*i+1 right(i) = 2*i+2

// STL priority_queue
priority_queue<int> maxHeap; // max-heap
priority_queue<int, vector<int>, greater<int>> minHeap; //
min-heap
priority_queue<pair<int,int>> pqPairs; // orders by .first, then .
second

// Custom comparator (lambda)
auto cmp = [](T a, T b) { return /* a has LOWER priority than b
*/; };
priority_queue<T, vector<T>, decltype(cmp)> pq(cmp);

// Core operations: .push(x) .pop() .top() .empty() .size()
// K largest of n: min-heap capped at size k -> O(n log k)
// Merge k sorted: min-heap of one front element per list -> O(N
log k)
// Running median: max-heap (lower half) + min-heap (upper half),
rebalance
```

Coming up in Part XIII

Part XIII — Graphs starts a large, self-contained section: representations (adjacency list/matrix), BFS and DFS, connected components, shortest paths (including a heap-powered Dijkstra that will feel immediately familiar after this Part), minimum spanning trees, and DSU. The heap intuition built here — “always expand the current best candidate” — is the backbone of several of those algorithms.